Power Prediction of Embedded Scalar and Vector Processor: Challenges and Solutions

Vijay Kiran Kalyanam, Peter G. Sassone
Qualcomm Technologies Inc., Austin, TX, USA
vijaykir,psassone@qti.qualcomm.com

Jacob A. Abraham
CERC, The University of Texas at Austin, USA
jaa@cerc.utexas.edu

***Abstract*—We present our methodology in applying a well established statistical dynamic power prediction technique in a production environment to an embedded commercial 'scalar and vector processor'. The pitfalls faced and solutions to guide the statistical solver to build a low error power prediction model are discussed. In our proposed method, we extracted stall probe-points in a processor, used selective microarchitectural events, created instruction groups and selected sub-set of performance events to refine the power-model.Our approach to determine the processor's dynamic power floor and power-sampling window size in an architectural trace and the selection of tests for training are explained. Our flow results in power weights for a set of architecturally visible events as well as few optional microarchitectural events of the processor. Using these weights, a canonical power prediction equation (that is configurable with user specified granularity of the abstraction of events) was auto-generated. On comparing the predicted-power results of our proposed method against the golden power numbers from a commercial EDA tool, we obtain an average-power error of 8% and reasonably track instantaneous power of *new* unseen real application workloads using lesser compute memory resources and achieving significant compute speedup.**

***Keywords*—Machine learning, power-event selection, architectural and micro-architectural power model.**

I. Introduction

Designing low power mobile/Internet of Things(IoT) processors used in embedded systems is becoming critical for its success. To achieve the low power goals, the design house or its customers need a way to measure the power very early in design cycle and also enable power-aware modeling in early microarchitecture design phase as discussed in [1]. But, if the customers are not willing to share their real software applications to be run on the embedded processor, it becomes a challenge to provide a power estimate. One solution for this problem is to build a power model that could be validated by annotating power during FPGA emulation or architectural simulation using instruction set simulator, while running big representative workloads. It could also be an added advantage to characterize and share the device power such that the customer can use embedded software's runtime execution trace to project power for their proprietary workloads. Additionally, voltage scaling and increasingly higher current demands pose challenges to various aspects of power-management. Since analog voltage droop detectors and current sensors are complex expensive circuits, digital power meters and digital voltage droop detectors provide viable cost effective alternatives. Hence, having an early power model that models both the instantaneous and average power will be valuable. Traditionally, power-calculators are built manually using highly error prone methods such as using approximations of a few workload characteristic parameters from the current design or using back of the envelope calculations.Another method used is simulation of a few sample application slices of a common software workload and obtaining its power estimate from an EDA power tool. This could introduce in-accuracies to the projected power, if the workloads chosen to build the model are not fully representative of real application use-cases of the embedded processor. We propose and build a statistical power prediction model for an embedded commercial Qualcomm(R) Hexagon(TM) 680 DSP with Hexagon vector extensions (HVX) and 512KB L2, shown in Fig. 1 that is inside a Qualcomm(R) Snapdragon(TM) SoC[1]. We estimate both average power and instantaneous power using design features that are visible though workload analysis like instructions or performance monitoring (PMU) events and guide this process using few additional microarchitectural events.

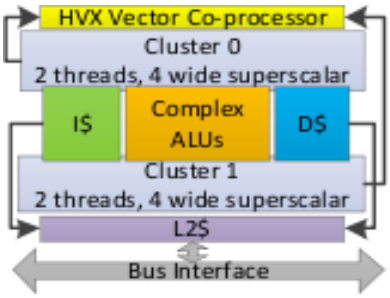


Fig. 1. **Embedded Scalar/Vector Engine**

II. Motivation and Related Work

In previous work, a framework for architectural level power analysis has been addressed in Wattch [2] and McPAT [3], both of which used power-characterized models of processor's functional building blocks. These methods are inapplicable to current state of the art processors due to physical design synthesis of the entire processor using standard cell libraries (except for some custom array structures using memory libraries) thus making it impractical to obtain pre-characterized

[1]Qualcomm Hexagon and Qualcomm Snapdragon are products of Qualcomm Technologies, Inc. Qualcomm Hexagon has shipped more than a billion devices to customers.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.

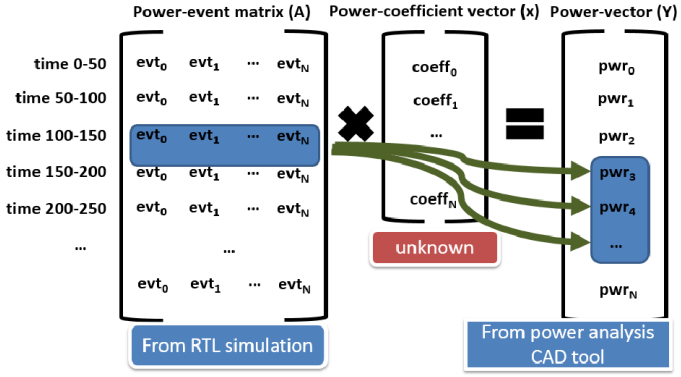


Fig. 2. **Power Event Matrix**

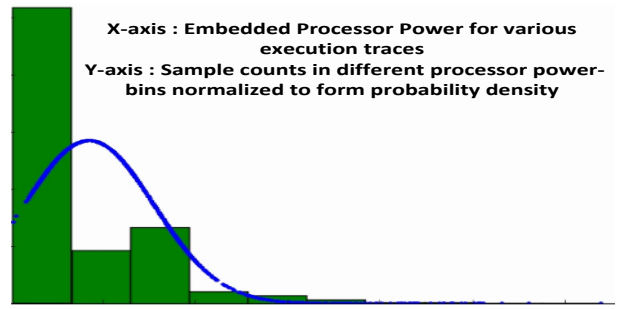


Fig. 3. **Power-binning and power-distribution of Execution trace samples**

power of functional sub-blocks. Wattch and McPAT also use simplified power models for memories, clocks, combinational logic which is error-prone. The non-machine learning approach used makes these methods potentially more error-prone to predict power of unseen benchmarks. Generation of statistical power models has already been proposed to processors in previous works [4], [5] and [6]. [7] discusses building a statistical power model using regression analysis, but only on simplest of data-path circuits like adders, multipliers. We will discuss our work and our contributions by comparing against these previous works in the following sections. In our work, we apply statistical techniques for power prediction and discuss the challenges encountered. This resulted in proposing solutions to problems not addressed in earlier works, related to a generic multi-threaded processor with potentially*a) large instruction set, b) many redundant PMU and some rare PMU events, c) multiple pipeline stall points, d) co-processor and external master/slave bus interfaces nearly representing the processor in a system*. An important distinction between our work and many others in this field is *we use functional coverage and performance verification programs for solving the power model, and real world programs for checking the power prediction results*. This topic of functional test selection has not been addressed in previous works.

III. Enabling Power prediction

The total power consumed by a processor is given by $P = dynamic\ power(\alpha * C * V^2 * F) + short\ circuit\ power(V * I_s) + leakage\ power(L)$, where the parameters of the processor α denotes its toggle activity, C is its switching capacitance, V is supply voltage, I_s is short circuit current and F is frequency of the clock. Our goal in this work is to predict the total power consumed by the processor (ignoring its leakage power (L)) while executing any software application trace. We approach this problem by modeling total power (P) subtracting leakage power (L) as a power prediction model of the form $Y = xA + K$. This equation models the relation between dependent variable (Y) and the coefficients (x) of regressors as linear through an error variable K. The goal is to determine x, given A and Y. This model could be solved using statistical linear regression algorithms [8]. Fig. 2, illustrates the representation of this model with a system of linear equations built with power-event matrix(A),power-coefficient vector(x) and power-vector(Y) denoting power consumed by the processor's execution trace. The coefficient of determination (R-squared) statistic is used to measure the prediction accuracy of the fit of the line to the data. Though, this looks straight forward, problems explained in section II and complexity of our processor explained in this section imposed challenges. Also the linear assumption may not be entirely true as power consumption could be non-linear on two or more events.

The embedded processor used in this study consists of a complex state of the art scalar in-order multi-issue very long instruction word(VLIW), whose previous generation design was presented in [9]) and HVX (a high compute-bandwidth vector co-processor [10]). As shown in Fig. 1, scalar core has two parallel super-scalar clusters, where each can execute up to 2-threads/4-instruction wide packet per cycle and upto 4-threads per cycle . The instruction/fetch unit (IU) consisting of instruction cache (I$), data unit (DU) with data cache (D$) and complex ALUs in the execution unit (XU) are shared between clusters, creating complex resource scheduling. The processor also supports simultaneous multithreading (SMT) for executing multiple parallel hardware threads. Also there are various buffers/queues and FIFOs as shown in Fig. 6, which can create many stall points, that could translate to wait periods of potentially no-execution in one or more pipeline stages. The HVX operates on large single instruction multiple data (SIMD) vector and is highly power hungry. Since the nominal power of HVX operating on 1024 bits of data, is higher than the maximum power of the scalar core, it is important to predict power correctly. Additionally, we have little architectural visibility into HVX from PMU events. The events that we do have are actually accounted for at a pipeline(pipe) stage that is different from the pipe stages where most of the HVX power is consumed. With HVX's execution occurring post commit, the vector co-processor's power profile is skewed with respect to the scalar processor's instruction commit. The power-binning and power-distribution of scalar and vector processor's execution traces are shown in Fig. 3. The skewed distribution shows that the high power contribution from HVX (towards the right of the x-axis in Fig. 3) on some scenarios. All these present complex picture of power usage across different pipe stages. Considering the complexity of the processor, our solutions are described next.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.

A. *Instruction Event Grouping*

[11] and [12] both show that instruction power estimates are useful for software power estimation. [11] proposes power characterization for every instruction which is prohibitively time-intensive and inefficient for any processor. This method becomes impractical for a processor with an enormous instruction set, such as ours. Instead of conventional instruction power modeling([11], [12]), which ideally needs all possible scenarios of instruction flow in the pipeline, we extended this concept to extract instructions during its execution as events and provided it as a power-event feature to train the statistical predictor. But, if a processor has many variants of instructions, then including an event for each of thousand different opcodes confounds the solver algorithms. This is because of the well known statistical requirement of the need to have an optimal sample count to feature ratio for learning algorithms to work well. For example, a low power instruction can obtain a high power co-efficient due to incorrect solution by the predictor. To solve this problem, instead of using events of unique instructions, we grouped the instructions based on functional categories and called these opclass (opcode-class) events.

Opclasses included unique categories for integer arithmetic, floating point, jumps, loads/stores, and co-processor instructions sub-classified as vector(loads,stores,arithmetic) instructions. This grouping significantly reduced the number of events that are being recorded to around 150 opclasses from around 1000 unique instructions and simplified the linear system of equations to be solved resulting in a predictor with improved accuracy. *Ours is a first attempt of providing opclass grouping for statistical power predictors*. Due to opclass grouping, we lost the ability to predict individual instruction power with high accuracy. This is because, unless we solve for accurate coefficients for those specific opcodes, we'll be off in prediction. Despite a high overall sample size, we did not have enough samples of every instruction, for the solver to obtain its predicted power correctly, in isolation from everything else going on in the system. So, our equation does not solve correctly for sparse instruction scenarios which is a practically rare case. Instead, our method does well for real-world big embedded software applications consisting of dense instruction execution streams. Also, we did not perform grouping based on various VLIW packet combinations as it was not necessary. We instead accounted for packet-decodes & packet-commit events described further in III-B. The importance of our opclass grouping approach is shown in Fig. 4 where the results on test data shows that choosing opclasses over instructions as the input events for the prediction power model improves the instantaneous power model error by around 4%, reducing the error obtained to 28.23% (from 32.54%) using opclasses (instead of unique-instructions).

B. *Pruned Short-PMU eliminating rare/redundant ones*

[11] and [12] represented inter-instruction effects by building an adhoc instruction power model using a small set of events like data-cache (D$) misses, instruction cache (I$) misses and NOPs which is not entirely representative of all

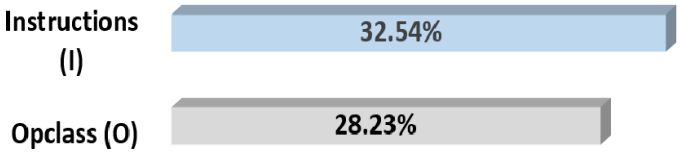


Fig. 4. **Power Prediction Model Error improvement using Opclass Events (Instruction grouping) instead of using unique-Instruction Events**

possible scenarios of instruction flow in the pipeline. Instead, our work characterizes various opclass's runtime execution traces in combination with selected PMU events. Being a multi-threaded VLIW machine, there are various PMU events representing cycle-level, thread-level, instruction-level, packet-level granularity, bus-interface PMUs and high-power vector co-processor PMUs. We extracted all 160 PMU events from the design and called it full PMU (P_f) events. The P_f count is 10X higher than the processor studied in [4]. This 10X statistic is important because the pitfall related to the P_f events were identified to be, not the lack of these events, but instead, presence of multiple and redundant PMUs. Also the presence of some rare PMU events was causing the solver to assign incorrect weights and needed appropriate elimination. The challenge we faced in our work is to eliminate these rare/redundant events. Rather than considering all PMU events (some of which are completely irrelevant to power modeling and available for interesting performance scenarios), only power-related PMU events could be selected. Encouraged by the results of grouping the instruction events down to opclasses, we attempted a similar trimming of the PMU events to form short-PMU (P_s) events.Unlike a simpler task in [4], where only 6 PMU events were hand picked from the original 15 PMU events in the list, in our work all P_f runs were analyzed and used as guidance in selection of P_s events. This process is further explained in sections III-B1, III-B2 and the heuristic in Algorithm 1. The main goal here was to get the solver to do a better job of assigning power-weights to the PMU events. Even if the linear assumption is approximately true, all PMUs cannot be provided to the solver to let it throw away the unuseful ones. By selecting P_s, we are indirectly eliminating dependent PMU variables that the linear solvers have difficulty in identification. For example, there could be various types of *commit* events in an execution trace. The solver pointed to one of the PMU *commit* events when provided with entire P_f list. We used solver's hint to pick *any/all commits* as an P_s event and removed many redundant commit events.

1) ***Determining Dynamic Power floor****:* The topic of determining the processor's dynamic power floor is not addressed in [4], [5]. This is not to be confused with leakage(idle) power, which is not considered in our work. We define the processor's dynamic power floor as power consumed by the processor when minimal set of functional sub-units have its small set of local clocks running (due to high clock gating efficiency of design) and is waiting for some asynchronous event to quickly enter high performance mode.Here the goal is to reduce the constant (intercept) in the linear power equation by performing multiple iterations as shown in the Table I and Algorithm 1. It

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ILLUSTRATION FOR CONVERGENCE ON FLOOR POWER

Iteration	1	2	..	N
Floor	5X	4X	2X	X
Error%	baseline+Y1+Y2+Y3	baseline+Y2+Y3	baseline+Y1	baseline
Events	cacheline-drops	no-dispatch	stall,..others	miss-cycles

involved picking right set of events for a) wasted power events like dropping of cache-lines, partial packets or flush events and b) idle or low-activity dynamic events. From the illustrative example shown in the Table I, at iteration count 1, the dynamic power floor was high (5X) resulting in error of Y1+Y2+Y3 more than the baseline dynamic power floor that is achievable in iteration N. Multiple iterations were performed as part of the P_s pruning that resulted in picking the right set of events that do-not represent high power but instead represent low/no-activity power. For example as shown in the Table I, the events picked during iterations included *no-dispatch* in iteration 2, *stall events, miss-cycles,* etc. Finally, the solver converged to an intercept value denoting rock bottom dynamic power floor.

*2) **Power-weights of VLIW and SMT properties**:* Being a VLIW machine, the burden is on the compiler to determine the scheduling of instructions.Also, since VLIW power is consumed in building a packet, and if there is no PMU event associated with it, its power is automatically learnt and associated with the equation's intercept.The power weights generated from our method are robust enough and agnostic to VLIW compiler optimization like loop-unrolling or vectorizing of the programs, since it is translated to executable machine code and eventually run by the processor. Being an SMT machine, our pruned P_s list did not include number of $hardware\ threads$ as an event, though it determines performance. The reasoning is, though different $threads$ in any software application could have different $instructions\ per\ packet$ (IPP) and different $packet\ rates$, pruned P_s events constituting selected PMU events like $fetch$, $commit$, $memory\ access$ events accounts for higher bandwidth enabled by more $threads$. Hence, these P_s events are more correlated to power than $threads$. For example,a 1-threaded program with a stream of back-skip-back (where a younger packet is $committed$ 2 cycles after previous older packet in same $thread$) packets every cycle should yield roughly the same power as a 2-threaded program in a single cluster with those same streams (still yielding 1 packet per cycle). So, we can conclude that for the power prediction model, the $packet\ rate$ is a 1st order event. The $thread\ count$ (contributing to the $packet\ rate$) is a lower order event.

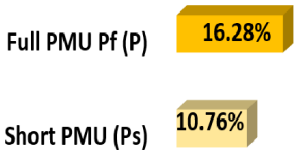


Fig. 5. **Instantaneous Power Model Error improvement by selecting Short-PMU events over using all PMU events**

To summarize, we use the heuristic in Algorithm 1 to select PMU events that results in a low error prediction model. The automation of this iterative process is a needed improvement. The instantaneous power model built with P_s has better accuracy (5% lower error) compared to model built using P_f as shown in Fig. 5, which is a testament to our pruning process.

Algorithm 1 Short-PMU Event Selection Procedure

Input: All the chosen power events of a processor: opclasses, full PMU P_f and micro-architectural events(N) **Output:** Selected set of short-PMU events P_s resulting in a low prediction error

1: **while** power prediction error is high, iterate the solver and select P_s in each iteration **do**
2: Using PMUs auto-selected by the solver as reference, select potential first order PMU events based on architectural/microarchitectural description.
3: Use the solver picked PMUs as a guide to pick any PMU events missed from design knowledge/description.
4: Pick the unique PMU events that are potential super sets and eliminate any of its equivalent subset PMUs picked by the solver. This also helps avoid picking PMUs that are redundant.
5: Choose PMU events that represent wasted power events, idle or low-activity dynamic events and iterate the solver towards a goal of obtaining lower intercept value(dynamic power floor) in the linear prediction equation.
6: **end while**

C. *Stall Points in statistical power model framework*

The data/control flow of the processor in this study shows queues, buffer/FIFO stall-points marked PROBEs (Fig. 6) in the processor, where an instruction can reside for dozens or hundreds of cycles. The challenge in power prediction enablement is to sample the right set of PROBEs. The opclasses discussed in III-A are picked from the one of the pipe-stages in the control unit(CU) shown by decode and dependency logic in Fig. 6. With multiple buffers, queues and stall-points, an instruction can take hundreds of cycles from fetch to commit, and power is consumed at several different points in different timeslices.To correctly attribute an instruction's power within the timeslice, we count the instruction power in three different stall points in its different pipe-stages- Icache(IU) delivery, decode post the packet queue(PQ) and the actual instructions sampled out of instruction queue(IQ). Another stall probe point within HVX was used as well where its vector fifo time-staggers the coprocessor's execution post-commit, from the time when the instructions are initially dispatched from the CU. *Ours is*

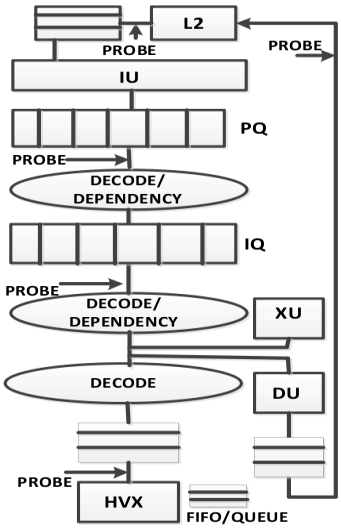


Fig. 6. **Buffers, FIFOs and Queues**

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.

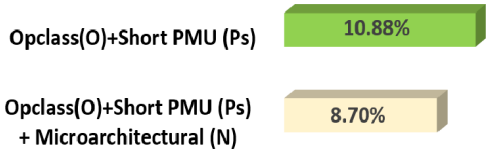


Fig. 7. **Instantaneous Power Model Error improvement using selective microarchitectural events**

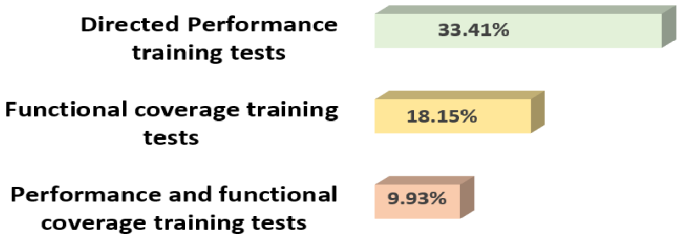


Fig. 8. **Average Power Model Error improvement using both directed performance tests and functional coverage closure tests**

first to use stall points for statistical power prediction. We call these additional probe point events $temporary\ events$ because it helps with better training of the model. To form the final power equation, the power weights across these $temporary\ events$ are then merged to the obtain the weight of the real event that is architecturally visible. For average power, it does not matter when the power was consumed in an instruction's lifetime. Probing at these stall points help in computing the correct instantaneous power model. The stall points identified for the power-hungry coprocessor were critical enough for the power model, that some of these stall points are being added as architecturally visible PMU events to be useful for power prediction in our next processor. The stall points we had used to predict a specific design's power is very different from the design parameters like instruction bandwidth, variable queue sizes, cache sizes, programmable latencies used in [6] for design-space exploration.

D. *Selective addition of Microarchitectural events*

A key insight we had in our work, is that, even if the final solution is to only have weights assigned to architectural events, it is still beneficial to train with small set of microarchitectural events (and later discard it for test data) to improve model accuracy. For example, during model refinement, microarchitectural events like very high power L2 cache (L2$) clear events showed up consistently due to its invocation by random training tests. This corrupted the solver, due to incorrect assignment of weights to opclass or P_s events that were active during the same time. Since there were no PMU events associated for L2$ clear, it was critical to create a $temporary$ microarchitectural event for this rare event (occurs only during processor bootup).Another microarchitectural event was related to turning on the high power hungry regional clock buffers (RCB), during the transition of the processor from the supported dynamic multithreading (DMT) mode to a legacy interleaved multi-threading (IMT) mode. The processor has a clock tree structure as described in [13]. Selectively adding RCB enable microarchitectural events as inputs to the solver, helps the training model to solve for right power-coefficients for real architecturally visible events.

As part of the power event selection we performed statistical correlation analysis, that helped identify data-set bugs due to random force toggling of RCB clocks. The RCB enablement makes it easier to exercise various design features for functional verification coverage. This resulted in big prediction errors(due to multiplicative and non-linear effect of RCB power on the clock tree), which was eventually fixed, by using right training-program stimulus. As real programs only turn on RCBs, when it is functionally valid, training tests were regenerated after removal of these random RCB clock toggles. Instead of our controlled addition of processor's internal microarchitectural signals (which are later discarded for average power model), [5] proposed using all possible internal processor signals, most of which are either non-PMU signals or not architecturally visible, hence cannot be used for architectural power model generation. The graph in Fig. 7 shows that we obtain a non-negligible 2% error improvement reducing the instantaneous power error down to 8.70% from 10.88% using very selective set of additional microarchitectural events. To emphasize, this is a key improvement in error because the test data stimulus did not enable these micro-architectural events such as L2$ and clocks (discussed earlier in this section) that caused high power. In other words, selective addition of these events made our power model more accurate. We would have seen a even higher error improvement if the test data did contain these L2$ clear and high clocking power RCB events.

E. *Selecting tests for training model*

Picking the small and complete set of training tests for power prediction is a challenge. To the best of our knowledge, most previous and related work seem to generate training dataset from randomly extracting 10% to 20% of their test dataset. *Since we believe the "blindness" of a solution to the test data shows the robustness of a characterization to any program that may be presented in the future, we used large collection of randomized and directed functional coverage tests (2000+ tests) and supplemented it with performance tests (200+) for our power model generation. All of these training tests are agnostic to test data. This has not been addressed previously*. When we initially used the directed performance tests targeting PMUs for training the model, the solver mis-predicted badly. This is because these tests lacked high instruction coverage, did not hit PMU events in different ways and had low power-toggle coverage for all logical submodules in the processor. On the other hand, training with only randomly generated functional coverage tests lacked the directedness of exercising the PMU events. So, we used a mix of both test categories during training phase. This enabled us to reduce the average power error of the prediction model on test data to 9.93% from 33.41% as shown in Fig. 8.

F. *Picking the rightly sized sampling-window*

Related work usually speaks of accuracy in predicting the average power of a workload, which is easier to predict

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.

than instantaneous power because averaging effects of over-predicting and under-predicting errors can balance out. When large applications are executed on a complex processor triggering hundreds of events, then the system of equations to solve gets complicated for any linear regression algorithm. This is because the canonical power-equation obtained, now needs to accurately model instantaneous power for every time slice of processor's operation. *Our solution to this problem is windowing of the training data into fixed size time slices of specific clock counts called power sampling-window. This has not been addressed in previous works.* But then, we are presented with the problem of determining the power sampling-window size. In a processor, with queues/stall points as discussed in III-C, power may not be consumed in a particular time-slice where the power-event is counted. As shown in Fig. 2, if an event occurred in time slice i, power of it could be consumed at times i,i+1,i+n.For example, an I$ miss PMU event gets registered as a power event, but the power consumption that eventually results in the L2 fill (caused by I$ miss event) could be many processor cycles later. As shown by the green arrows in the Fig. 2, the power-events at time 100-150, could result in power-contribution of not just pwr2 corresponding to its time slice but could also contribute to pwr3 and pwr4. The power sampling-window size could vary dependent on the processor pipeline depth.

To determine the optimal power sampling-window size, we performed a sweep of training sets by varying the time slice (25,50,100,200 cycles) and applying it on 25(blue),50(green),100(black),200(yellow) cycle wide instantaneous test set and entire test's window (red) for average power as shown in Fig. 9. From this sweep, a curve was obtained as in Fig. 9 from which the size of the window/time-slice was picked to be 50 clocks(wide) for training data to obtain the power-coefficients as it had the lowest error. The coefficients obtained from training sample size of 50 clocks was then applied on various window sizes of test data. Fig. 9 shows that the least error was obtained for instantaneous power prediction with test window granularity of 100 clocks(black) whose error is also close to average power error(red) for entire test. This process provides users an option to pick the desired power sampling-window size granularity for instantaneous power prediction model based on the prediction error that could be tolerated. For the instantaneous power test results (Fig. 12), we chose the test window granularity of 100 clocks. Note that picking 200 clocks(yellow) for test would have resulted in a similar error but with worser power sampling-window granularity. Our method to choose the window size is not to be confused with trace-sampling problem addressed in [6] which was for identifying similar phases in a program. Also our prediction model inherently models the event ordering within the time window as well as both instruction/operand data dependencies by auto "learning" it from our large training set, though this topic is a potential area of improvement.

G. *Choosing the solver*

The matrix with events and power specified as shown in Fig. 2 was run through multiple linear regression algorithms [14] like NNLSTSQ, CVLASSO, ELASTICNET, LARS, LASSO, LSTSQ, RIDGE. Early in our work, when we had in-sufficient power-events and had corrupted/incorrect training data, results for runs attempted with these linear solvers showed a similarity of very high error of 32-37% and very low Rsquared (Adjusted Rsquared) value of 0.20-0.33 (0.29-0.33). In other words, none of the solvers perform well with bad training data. Also all solvers generated almost similar low-errors during re-runs after fixing the power-data integrity issues and including the missing power events. Hence, we conclude that the choice of linear regression algorithm is a secondary factor in power prediction accuracy at most, but for simplicity we use CVLASSO (Cross validated least absolute shrinkage and selection operator) [14] for power-modeling in our work. The CVLASSO algorithm does an internal cross validation, which is a key to not overfit the data, thus avoiding the methodological mistake of *learning the power weights of the power model and re-testing on the same data*. This algorithm generates both negative and positive coefficients during its solve. [5] generated power model equations that included negative coefficient terms. One disadvantage of using negative coefficients is the non-intuitiveness of an event's negative contribution to power. Hence, for simplicity of use in a production environment, we constraint and guide the solver to iterate to positive only coefficients.

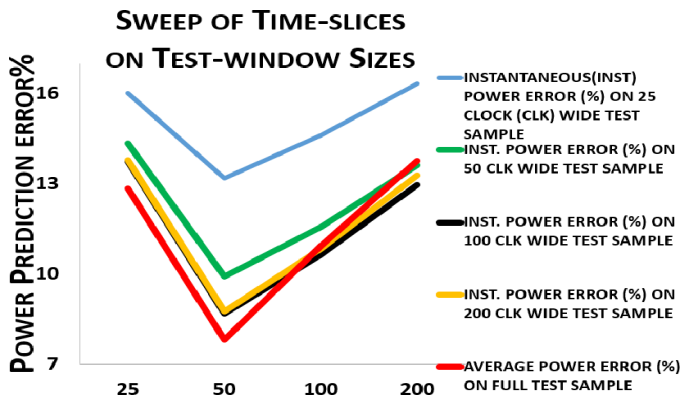


Fig. 9. **Power sampling-window size determination**

IV. Experimental Setup

Our production flow can be broken down into data collection, linear solving, and prediction check. For data collection, training and test data was collected. Each training program is run through the register-transfer-level (RTL) model of the embedded processor and waveforms of all signals was obtained using Synopsys verilog compiler simulator(VCS). This waveform was run through an EDA RTL power tool, Ansys PowerArtist and time-based instantaneous power was obtained. This tool does a quick synthesis of RTL and uses macro models for the memories and standard cell libraries. We could also apply our methodology on a gate-level power analysis sign off tool for an improved power equation accuracy. We also extract signals from RTL waveforms to generate power-event traces for training the predictor. The test data is a collection

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.

TABLE II
COMPARISON OF OUR WORK OP_sN AGAINST OTHER RELATED WORKS

Are the below techniques proposed in a statistical power prediction framework?	[2], [3], [4], [5], [7]	Our Work **OP_sN**
Instruction grouping and short-PMU selection	No	Yes
Extracting processor stall points for improving prediction	No	Yes
Selective Microarchitectural events to guide architectural power-weights	No	Yes
Functional test selection for training	No	Yes
Power sampling-window size determination	No	Yes
Analysis with regression solvers	No	Yes

(around 15) of gcc/llvm compiled code of real-world applications(apps) of our mobile processor which is predominantly an integer processor (with some floating-point support). The apps that we validated include the following : Dhrystone(dhry) 1-thread(1T)/4-threads(4T) is close representation of our system integer programming, Coremark is close representation of our CPU processor performance,Spec2000 MCF is for integer arithmetic, CRAFTY is for 64-bit word logical/branch prediction operations, JPEG/Feature Extract/VP8 decode/Denoise are used for videocoders/image processing, SIMD FIR filter 2T/4T for multi-threaded SIMD signal processing, memory copy represents bus transaction from L2 to external memory. Convolution 3x3 is for image processing and PDAF (phase detection autofocus) is a HVX camera benchmark. All of these apps are reasonable representations of target embedded apps potentially running on this processor. For checking our power characterization, the power-weights generated by the statistical solver is dot-producted against the test's power-event counts. This gives us predicted power for the test data. The prediction error is obtained by comparing the dynamic power results obtained from our proposed methodology against the golden power numbers from the commercial EDA (CAD) RTL power tool. We used open source tools like sklearn [14] and scipy [15] for statistical analysis and processing of our data. Leakage power(L) is ignored in our work, and could be roughly estimated by obtaining an area estimate and run-time temperature of the processor. The final predicted power of the processor could be obtained by adding L to our predicted dynamic power. Assuming availability of sufficient number of EDA tool licenses, running all 2000+ tests with the EDA tools sequentially, took approximately 2400 load sharing facility(LSF) machine hours.We could improve the flow's runtime by attempting trace-sampling [6] to identify phases of programs in training sets that behave similarly, thus reducing number of samples in training set. The real apps used for validation of our flow were simulated after it was sliced to multiple smaller programs using [16].

V. RESULTS AND CONCLUSION

As shown in Table II, while differentiating ourselves from previous work, we have discussed solutions to challenges of enabling power prediction flow in a production environment of a commercial embedded scalar and vector processor. We call our proposed prediction model OP_sN as it uses a combination of opclass(O), short-PMU(P_s) and incrementally added microarchitectural (or non-architectural) events denoted

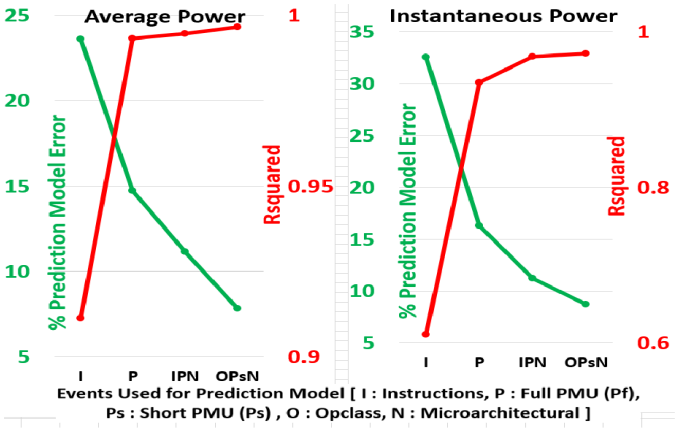


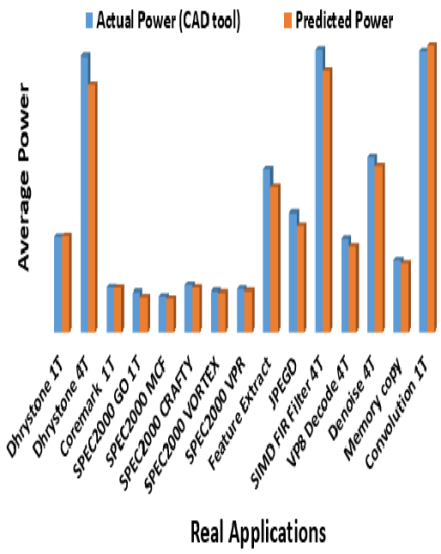
Fig. 10. **Improvement achieved in our work,OP_sN**

by N. This particular combination of events for the predictor was chosen because of the efficacy of each of the proposed solutions and improved model accuracy as summarized below:

- 4% prediction accuracy improvement obtained in using opclass(O) over instruction(I) events.
- 5% prediction error reduction achieved in using pruned short-PMUs (P_s) over full set of PMU events (P_f or P).
- Atleast 2% modeling accuracy improvement in using microarchitectural events (N) compared to not using N.
- Around 10-20% improvement by using both a)directed performance tests and b)random functional coverage tests for training the OP_sN model, versus using only either of (a) or (b).
- After choosing OP_sN model, an additional 3-6% improvement is achieved by picking the 50 cycle time window slice for training the predictor over picking a 25,100,200 cycle time slice.

We also showed that reduction of high sample-to-event ratio and the quality of data set contents are critical in obtaining a highly accurate power model. Fig. 10 compares the results of OP_sN model against prediction models that use only instructions(I) (or) only full set of PMU events (P_f or P) (or) a combination of I and P along with microarchitectural(IPN) in a statistical framework. The power event matrix size (specified in event columns by sample rows) for I, P, IPN, OP_sN matrices are respectively of approximate sizes of 1100 by 325K, 160 by 325K, 1170 by 325K, 190 by 325K.Fig. 10 shows that having every unique instruction (I) as the only predictor, results in a power model error (in green) of 23.61% for average power and 32.54% instantaneous power. Using full set of PMUs (P_f or P) only as predictor, results in an error of 14.71% for average power and 16.28% instantaneous power error. Using microarchitectural events (N) in addition to I and P reduces the error to around 11% error. By using OP_sN model, we have been able to improve the Rsquared statistic significantly (from 0.61 to 0.97 for instantaneous power) and also reduce the error of average and instantaneous power further down to around 7.83% and 8.7% respectively. Unlike [4], we see from the results that using only PMU events is in-sufficient for building an accurate statistical power model. This is validated with a 7-8% improvement between model generated using OP_sN over model generated with only P (P_f) events. We conclude that though PMUs are more important than instruction only events in a statistical prediction framework, we need combination of both as shown in OP_sN to obtain even lower

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.



Application	Error(%)
Dhrystone 1T	1
Dhrystone 4T	-10
Coremark 1T	-1
SPEC2000 GO 1T	-12
SPEC2000 MCF	-6
SPEC2000 CRAFTY	-6
SPEC2000 VORTEX	-4
SPEC2000 VPR	-5
Feature Extract	-10
JPEGD	-11
SIMD FIR Filter 4T	-7
VP8 Decode 4T	-9
Denoise 4T	-5
Memory copy	-5
Convolution 1T	2

Fig. 11. **Results of Real Application's Predicted Average Power using OP_sN model.**

error (Fig. 10).For the average dynamic power model (could be used for architectural level power calculator) generated out of OP_sN model, the power weights assigned to any microarchitectural(N) events are eventually either discarded or its weights merged with appropriate architecturally visible events. But while generating the instantaneous power model (could be used to build digital power meter) we do not ignore any terms in the OP_sN equation. The bar graphs of Fig. 11 shows average power (on y-axis) measured for running each of the real apps (along x-axis). Fig. 11 shows results obtained by CAD tool (in blue) versus the predicted power from our OP_sN method (in orange). It shows our average power error ranges between -12%(under prediction) to +2%(over prediction). The Fig. 12 shows the time of execution trace of the processor along x-axis and the instantaneous power averaged every 100 cycles along the y-axis for a sample set of test apps. The instantaneous power waveforms obtained from CAD tool(in blue) and predicted power with our OP_sN method(in orange) are plotted and superimposed to show how both track each other. From the Fig. 12, we show that we do quite well at instantaneous power which has less room for error. We conclude from the results that our power-model is reasonably accurate in predicting both average power and instantaneous power on real applications. This is especially significant given our experimental setup of using disjoint training programs and test programs unlike many of the related works. The compute time to calculate power using our prediction model is orders of magnitude faster and uses lesser compute memory compared to the EDA RTL power tool as shown in the logarithmic-scale graph in Fig. 13. This gives us confidence that our flow is both scalable and robust to predict power on *new* unseen real embedded software applications in a production environment. Finally, we hope our methodology and solutions proposed could be applied to other embedded-processors.

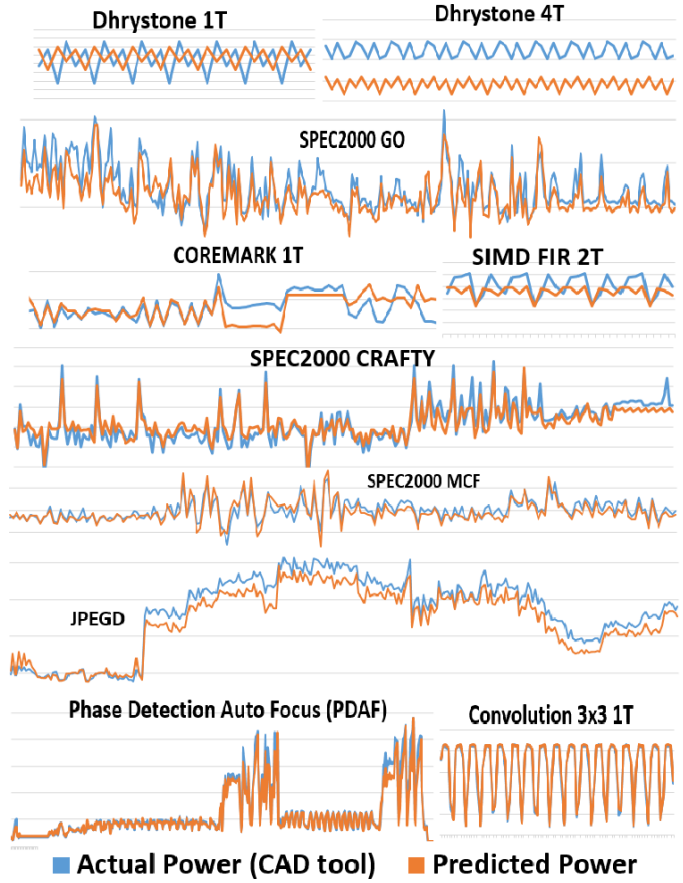


Fig. 12. **Results of Real Application's Predicted Instantaneous Power (y-axis: power, x-axis: time every 100 cycles) using our OP_sN model.**

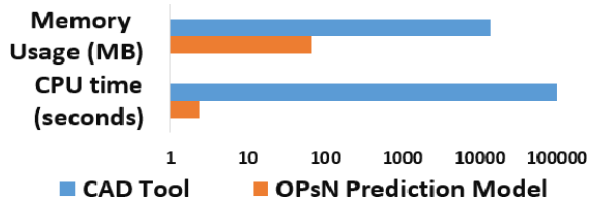


Fig. 13. **Power calculation compute time and memory usage using OP_sN model.**

References

[1] D.Brooks et.al, "Power-aware microarchitecture: design and modeling challenges for next-generation microprocessors," IEEE Micro, 2000.

[2] Martonosi et.al, "Wattch: A framework for architectural-level power analysis and optimization," ISCA, 2000.

[3] Jouppi et.al, "McPAT : An Integrated Power, Area, and Timing Modeling Framework for Manycore and Multicore Architectures ," Micro, 2009.

[4] Martonosi et.al, "Power prediction for Intel Xscale core using pmu events," ISLPED, 2005.

[5] R.Eickemeyer et.al, "Abstraction and microarchitecture scaling in early-stage power modeling," HPCA, 2009.

[6] D.Brooks et.al, "Accurate and efficient regression modeling for microarchitectural performance and power prediction," ASPLOS, 2006.

[7] Micheli et.al, "Regression-based RTL Power Modeling," TODAES, 2000.

[8] D. Freedman, "Statistical Models: Theory and Practice, " Cambridge University Press, 2009.

[9] L.Codrescu, "Hexagon DSP : An architecture optimized for Mobile multimedia and communications," Micro, 2014.

[10] L.Codrescu et al, "Architecture of the hexagon 680 DSP for Mobile imaging and computer vision," HotChips, 2015.

[11] S.Malik et.al, "Power analysis of embedded software:a first step towards software power minimization," TVLSI, 1994.

[12] Zafalon et.al, "Power modeling and estimation framework for VLIW embedded systems," PATMOS, 2001.

[13] M.Saint-Laurent, et.al, "A 28nm DSP powered by an on-chip LDO for high-performance and energy-efficient mobile applications," ISSCC,2014.

[14] E.Duchesnay, "Scikit-learn: Machine learning in Python," *JMLR*, 2011.

[15] E.Jones, "SciPy: Open source scientific tools for Python," 2001–.

[16] T.Sherwood et.al, "Automatically characterizing large scale program behavior," ACM SIGARCH Computer Architecture News,2002.

Authorized licensed use limited to: Hong Kong University of Science and Technology. Downloaded on April 05,2025 at 02:25:46 UTC from IEEE Xplore. Restrictions apply.